static, final and Initialisation Order

9 min read Updated 28 Jul 2026 Topic 11 of 65

Bookmark Mark done

Why it matters

`static` and `final` are two of the most-used keywords in Java and two of the most loosely understood. "Final means constant" and "static means global" are both close enough to be dangerous. The initialisation-order question that follows from them — what runs when — is a staple "what does this print?" puzzle, and the answer explains real problems: why a singleton can deadlock, why `final` does not make an object immutable, and why a static field is such a reliable memory leak.

The concept, simply

`static` means "belongs to the class, not to any instance". One copy exists, shared by every object, and it exists whether or not any object has been created.

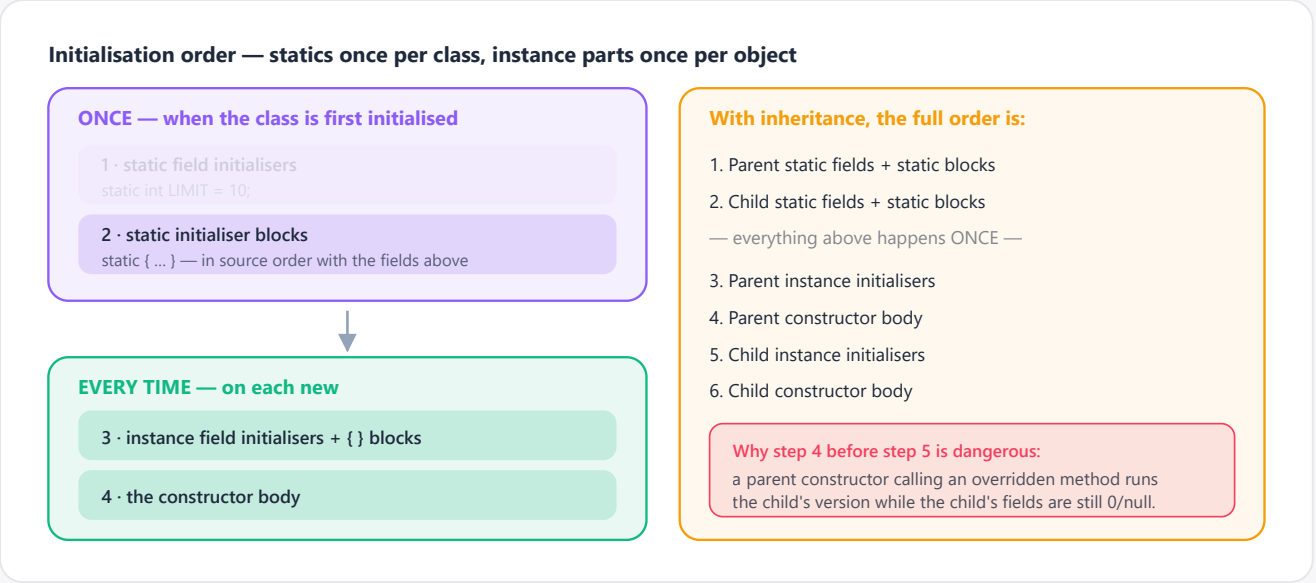
`final` means "assign exactly once". What it forbids depends on where it is:

Applied to	Means
A variable or field	May be assigned once; the *binding* cannot change (the object it points at still can)
A method	Cannot be overridden
A class	Cannot be extended
A parameter	Cannot be reassigned inside the method

`list = somethingElse` and permits `list.add("x")` all day.

How it works

Initialisation happens in two phases with a strict order:



Statics run once, when the class is first initialised. Instance parts run on every `new`. With inheritance, all statics precede all instance work, and the parent's instance phase precedes the child's.

Within each phase, **field initialisers and initialiser blocks run in source order** — a static block placed above a static field runs before it, which is occasionally surprising and a reason to keep them near each other.

When is a class initialised? Lazily, on first active use: instantiating it, calling a static method, or reading a non-constant static field. Notably it is *not* triggered by reading a `static final` compile-time constant — those are inlined into the caller's bytecode, so the class may never load at all. That inlining has a real consequence: change a public `static final int` in a library and every class that used it keeps the old value until it is recompiled.

`static` **methods** cannot use `this` or access instance members, cannot be overridden (only hidden — Chapter 2's earlier topic), and are dispatched from the declared type. **Static nested classes** matter more than they look: a non-static inner class holds an implicit reference to its enclosing instance, which is a classic leak — the

outer object cannot be collected while the inner one lives. Make nested classes `static` unless they genuinely need the outer instance.

Code examples

The order question, in the form it is usually asked:

JAVA Copy

```java
class Parent {
    static { System.out.println("1 parent static block"); }
    { System.out.println("3 parent instance block"); }
    Parent() { System.out.println("4 parent constructor"); }
}

class Child extends Parent {
    static { System.out.println("2 child static block"); }
    { System.out.println("5 child instance block"); }
    Child() { System.out.println("6 child constructor"); }
}

new Child();
new Child();
// 1 parent static block      ← once
// 2 child static block       ← once
// 3 parent instance block
// 4 parent constructor
// 5 child instance block
// 6 child constructor
// 3 / 4 / 5 / 6 again        ← statics do NOT repeat
```

`final` not meaning immutable:

JAVA Copy

```java
final List<String> names = new ArrayList<>();
names.add("Asha");          // ✓ the OBJECT is mutable
names.add("Ben");           // ✓
// names = new ArrayList<>(); // ✗ the BINDING is fixed

// Actual immutability is a property of the object, not the variable:
```

The three flavours of `final` field, which behave quite differently:

JAVA Copy

```java
public class Config {
    // 1. Compile-time constant — INLINED into every caller's bytecode.
    public static final int MAX_RETRIES = 3;

    // 2. Static final, computed at class init — NOT inlined, so changing
    //    it takes effect for existing callers without recompiling them.
    public static final String HOST = System.getenv("APP_HOST");

    // 3. Instance final — one per object, assigned in the constructor.
    private final Instant createdAt;

    public Config() {
        this.createdAt = Instant.now();     // ✓ blank final, assigned onc
    }
}
```

And the static-field leak, which is the practical consequence people meet in production:

JAVA Copy

```java
public class Cache {
    // A static field is a GC ROOT. Anything reachable from it lives as lo
    // as the class does — which is usually as long as the application.
    private static final Map<String, byte[]> CACHE = new HashMap<>();

    public static void put(String k, byte[] v) { CACHE.put(k, v); }
    // No eviction, no bound, no removal → an OutOfMemoryError with a
    // perfectly healthy-looking code path. Chapter 1's "reachable is not
    // useful" in its most common real form.
}
```

Under the hood

Class initialisation is thread-safe, and the JVM guarantees it. The static initialiser runs exactly once, under a lock the JVM holds internally. That guarantee is what makes the holder idiom the cleanest lazy singleton in Java — no `synchronized`, no double-checked locking, no `volatile`:

JAVA Copy

```
public class Singleton {
    private Singleton() { }

    private static class Holder {                // not initialised until
        static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;                  // …this line first runs
    }
}
// Holder is only initialised on first access to getInstance(), and the JV
// serialises that initialisation. Lazy AND thread-safe, for free.
```

The same lock is also a hazard: if a static initialiser blocks — waiting on another class that is itself waiting on this one — you get a **class initialisation deadlock**, which is genuinely hard to diagnose because the stack traces look idle. Keep static blocks trivial.

`final` **fields have memory-model meaning**, which is the senior-level answer here. The Java Memory Model guarantees that any thread which sees a properly constructed object also sees its correctly initialised `final` fields — *without* synchronisation. This is why immutable objects are safely publishable across threads, and it is one of the strongest arguments for making fields `final` by default. The guarantee is void if `this` escaped during construction. Chapter 8 develops this.

Common mistakes

- "`final` **means immutable.**" It fixes the binding only. Use immutable types for immutable objects.

- **Using `static` as a global variable.** Shared mutable state with no owner — untestable and a race waiting to happen.
- **Unbounded static collections.** Static fields are GC roots. This is the archetypal Java memory leak.
- **Assuming a public `static final` constant can be changed without recompiling callers.** Compile-time constants are inlined.
- **Non-static inner classes by default.** They pin the enclosing instance in memory. Make them `static` unless they need the outer object.
- **Heavy work in a static block.** Slows first use, can deadlock, and failures surface as a confusing `ExceptionInInitializerError` (and `NoClassDefFoundError` on every later attempt).
- **Thinking `static` methods are overridable.** They are hidden and dispatched from the declared type.

Interview lens

Q: "What does `static` mean?"

A: The member belongs to the class rather than to any instance — one copy, shared, available without creating an object. Static methods cannot use `this` or touch instance state, and they are hidden rather than overridden in subclasses. Static fields live with the class metadata and act as GC roots.

Q: "Does `final` make an object immutable?"

A: No. It makes the *reference* unchangeable. `final List<String> l = new ArrayList<>()` forbids reassigning `l` and permits `l.add(...)`. Immutability is a property of the object's design — no mutable state and no setters — which is what `List.of(...)` or a record gives you.

Q: "What is the initialisation order?"

A: All statics first and once: parent static fields and blocks, then child's, in source order. Then per instance: parent instance initialisers, parent constructor body,

constructor sees null fields.

Q: "When is a class initialised?"

A: Lazily, on first active use — instantiation, a static method call, or reading a non-constant static field. Reading a `static final` compile-time constant does not trigger it, because the value was inlined into the caller at compile time.

Q: "Is static initialisation thread-safe?"

A: Yes — the JVM guarantees a class's static initialiser runs exactly once, serialised under an internal lock. That is what makes the initialisation-on-demand holder idiom a correct lazy singleton with no explicit synchronisation. The same lock means a static initialiser that blocks on another class can deadlock.

Q: "Why prefer `final` fields?"

A: Two reasons. Design: the object cannot drift into an inconsistent state after construction. Concurrency: the memory model guarantees that any thread seeing a properly constructed object also sees its `final` fields fully initialised, with no synchronisation — which is what makes immutable objects safe to share. That guarantee is forfeited if `this` escapes during construction.

Q: "Can a static method be overridden?"

A: No — redeclaring it in a subclass hides it. Statics are resolved from the declared type at compile time, so `Parent p = new Child(); p.staticMethod();` runs the parent's. Calling a static through an instance reference is legal but misleading; call it on the class.

Cheat-card

- `static` = per class, one copy, no `this`, hidden not overridden, **GC root**.
- `final` = assign once. Field → binding fixed. Method → no override. Class → no extend.
- `final` **≠ immutable.** The object stays mutable.
- Order: parent statics → child statics (once) → parent instance init → parent ctor → child instance init → child ctor.
- Within a phase, **source order** decides.
- Class init is **lazy and thread-safe** → holder idiom = the clean lazy singleton.
- `static final` compile-time constants are **inlined** into callers.
- `final` fields carry a **JMM safe-publication guarantee** — void if `this` escapes.
- Make nested classes `static` — non-static inner classes pin the outer instance.

QUICK CHECK — no marks, just for you.

1. final List<String> list = new ArrayList<>(); Which operation fails to compile?

- ○ list.add("x")
- ○ list.clear()
- ○ list.remove(0)
- ○ list = new ArrayList<>()

Check answer

2. Creating two instances of a class whose parent and child each have a static block and an instance block — how many times does each static block run?

- ○ Once each, when the class is first initialised — statics do not repeat per instance
- ○ Twice each, once per instance

- It depends on the constructor used

Check answer

3. Why is the initialisation-on-demand holder idiom a correct lazy singleton with no synchronized keyword?

- Because static fields are volatile by default
- Because the JVM guarantees a class's static initialiser runs exactly once, serialised under an internal lock, and the holder class is only initialised on first access
- Because the JIT inlines the getter
- It is not correct — it needs double-checked locking

Check answer

‹ Previous
equals() and hashCode(): The Contract

Next ›
Why String Is Immutable

LEARN MORE

ARTICLES & DOCS

- Initialization of classes and interfaces (JVMS §5.5)
  docs.oracle.com
- Static initializers — The Java Tutorials
  docs.oracle.com

VIDEOS

Java Interview Kit

"static final initialization order" — video explainers

YouTube

static final initialization order interview questions

YouTube

Hand-picked references for this lesson.